```
    @FindBy(name="password")
    WebElement password;

    public LoginPage(WebDriver driver) {
        PageFactory.initElements(driver, this);
    }

    public void signinWithCredentials(String userEmail,
                                      String userPassword) {
        email.sendKeys(userEmail);
        password.sendKeys(userPassword);
    }
}
```

❷ **Look up the password field.**

❸ **Initialize the email and password fields.**

❹ **Look up the web elements on the web page before using them.**

The `@FindBy` annotation tells WebDriver how to look up a `WebElement` field ❶, ❷. When you mark fields this way, you can use the `PageFactory.initElements()` method in the constructor ❸ to instantiate these fields for you. Each time you use these fields ❹, WebDriver performs the equivalent of a `driver.findElement(By...)` call to bind them to the corresponding element on the web page. The `@FindBy` annotation supports all of the different selector methods available when you use `driver.findElement(By...)` (see table 8.4).

Table 8.4 Different ways to use the `@FindBy` annotation

`@FindBy` expression	Description
`@FindBy(id="welcome-message")`	Find by ID
`@FindBy(name="email")`	Find by name
`@FindBy(className="typeahead")`	Find by CSS class name
`@FindBy(css=".typeahead li")`	Find by CSS selector
`@FindBy(linkText="Book")`	Find by link text
`@FindBy(partialLinkText="Book")`	Find by partial link text
`@FindBy(tagName="h2")`	Find by HTML tag
`@FindBy(xpath="//span[.='Singapore']")`	Find by XPath expression

If the name of the WebElement fields in your page object matches either the name or ID of the corresponding HTML element, you can skip the `@FindBy` annotation entirely:

```
public class LoginPage {

    WebElement email;
    WebElement password;

    public LoginPage(WebDriver driver) {
        PageFactory.initElements(driver, this);
    }
    ...
}
```

❶ **You don't need the @FindBy annotation here.**